LECONS :

Clé étrangère : foreign key

Structure

Ajouter

alter table PRODUIT add column POIDS smallint;

Supprimer

alter table PRODUIT drop column PRIX;

Modifier valeur par défaut

alter table CLIENT modify column CAT set '00';

Supprimer valeur par défaut

alter table CLIENT modify column CAT drop default;

MODIFICATION TABLE : Contraintes
(règle hasstraka)

alter table PROSPECT add primary key (NCLI);

alter table CLIENT add unique (NOM, ADRESSE, LOCALITE);

alter table CLIENT modify CAT not null;
alter table CLIENT modify ADRESSE null;

alter table CLIENT
add foreign key (CAT) references CATEGORIE (CAT);

te hi-afficher karazana izay ity misy repetition dia mampiasa DISTINCT.

AS (ALIAS)

Fanomezana anarana colonne vaovao

(ex select adi as identifiant from client ;)

Where

ex Select nom from client where adi = 4 ;

Le commande where dans une requête SQL permet d' extraire les lignes d'une BD que respectent les indices.

Delete

La commande Delete en SQL permet de supprimer des lignes dans table

Syntaxe

DELETE From table
Where condition.

- Insérer une ligne en spécifiant seulement les colonnes souhaités (colonnes obligatoires)

Insert into TOTAL
(nom colonne 1, nom colonne 2, ...) ~~values~~
VALUES ('valeur 1', 'valeur 2', ...)
exemple

Insert into client (ncli, nom, adresse, localite, compte, datenaiss) values (3, 'RAVAO', 'Lot 253/3306', 'Antananarivo', '22222', '1993-04-28');

Select
L'utilisation la plus courante de SQL consiste à lire des données issues de BD.

Syntaxe
Select ~~from~~ nom_du_champ
FROM nom_du_Tableau

ex
SELECT nom from Client;

LIKE

L'operateur LIKE est utilisé dans la clause WHERE des requêtes SQL.
Ce mot-clé permet d'effectuer une recherche sur un modèle particulier

```
SELECT *
FROM table
WHERE colonne LIKE modele
```

IS NULL / IS NOT NULL

Dans le langage SQL, l'operateur IS permet de filtrer les résultats qui

```
SELECT *
FROM 'table'
WHERE nom_colonne IS NULL
```

FONCTIONS DE STATISTIQUE

AVG() calcule la moyenne d'un set de valeur

COUNT(): pour compter le nombre de lignes concernées.

MAX(): pour recuperer

GROUP BY

Pour grouper leurs resultats et utiliser une fonct° de totaux sur un grp de résultat.

Syntaxe

```
SELECT colonne 1, fonction(colonne2)
FROM table
GROUP BY colonne 1
```

HAVING

La condit° HAVING en SQL est presque similaire à WHERE à la seule différence que HAVING permet de filtrer en utilisant des fonctions tel que SUM(), COUNT(), AVG(), MIN(), MAX().

EXERCICES :

DEVELOPPEUR

```
Create database gestion_projet ;
Create table DEVELOPPEUR (
    NumDev int (5), primary key, not null,
    NomDev char(30), not null,
    AdrDev char (50), not null,
    EmailDev char (50), not null,
    TelDev ~~int~~ ( 10), not null,
    );        Char

Create table PROJET (
    NumProj int (4), primary key, not null,
    TitreProj char (20), not null,
    DateDeb date not null,
    DateFin date not null,
    );

Create table LOGICIEL (
    CodeLog int (3), primary key, not null,
    NomLog char (20), not null,
    PrixLog ~~int (3) Prix~~ decimal (5, 2), not null,
    Foreign key (NumProj) references PROJET (NumProj) ) ;
```

```
Create table REALISATION (
    foreign key (NumProj) references PROJET (NumProj),
    foreign key (NumDev) references DEVELOPPEUR (NumDev)
    );

Create table REALISATION (
    NumProj  int (3), not null,
    NumDev   int (3), not null );


alter table LOGICIEL add column Num Proj int(3)
  not null;
```

Soit le schéma relationnel suivant:

Departement (NomD, NDep, Directeur)

Employe (Matricule, Nom, Prénom, DateNaissance, Adresse, Salaire, # N_dep, supérieur)

Projet (NomP, NPro, Lieu, # N_Dep)

Travaille (# Matricule, # NProj, Heures)

L'attribut superieur dans le relat° Employe contient le matricule du supérieur direct de l'employé

Chaque.

```
Create table DEPARTEMENT (
    NomD
```

```
Create table DEPARTEMENT (
  NDep int (3), primary key, not null,
  NomD char (30), not null,
  Directeur char(30), not null,
  );
```

```
Create table EMPLOYE (
  Matricule int (3), primary key, not null,
  Nom char (30), not null,
  Prenom char (30), not null,
  DateNaissance date, not null,
  Adresse char (20), not null,
  Salaire decimal (6, 3), not null
  );
Create table EMPLOYE (
  NDep int (3), not null

  );

Create table PROJET (
  Npro int (3), primary key, not null,
  NomP char (20), not null,
  foreign key (NDep) references DEPARTEMENT (NDep),
  );

Create table TRAVAILLE (
  Matricule int (3), not null,
  Npro int (3), not null,
  Heure Heure, not null
  );
```

```
Create Database GESTION_TRAVAILLE ;

Create table DEPARTEMENT (
    NomD varchar (32) , not null ,
    NumDep int (3) , not null ,
    Directeur varchar (20), not null ) ;
```

a faromy vo soratana, vita ny table retra²,

```
alter table DEPARTEMENT add primary key(NumDep);


Create table EMPLOYE (
    Matricule int (3) not null,
    Nom char (30) not null,
    Prenom char (20) not null,
    DateNaissance date not null,
    Adresse varchar (20) not null,
    Salaire int ( 5, 2 ) not null,
    Superieur char ( 10) not null
```

```
alter table EMPLOYE add primary key (Matricule);
alter table EMPLOYE add column NumDep int(3) not null;
alter table EMPLOYE add FOREIGN KEY (NumDep) references
    DEPARTEMENT (NumDep)

Create table PROJET (
    Numproj int(5);
    Lieu char (50),
    N Dep int
```